

rocq-robot: Enabling LLM-Driven Theorem Proving via Model Context Protocol

Gavin Mendel-Gleason
Scidonia, Dublin, Ireland
gavin@scidonia.com

May 26, 2026

Abstract

We present **rocq-robot**, a production-ready tool that bridges Large Language Models (LLMs) and the Rocq proof assistant (formerly Coq) through the Model Context Protocol (MCP). By exposing proof state management, tactic execution, and knowledge search capabilities through 20+ MCP tools, **rocq-robot** enables LLMs to autonomously construct formally verified proofs. The tool features project-aware workspace management, speculative execution for safe tactic exploration, and comprehensive error recovery. We demonstrate its effectiveness through one-shot autonomous proofs of a type preservation theorem for PCF with references using both DeepSeek v4 and Claude Sonnet 4.5—a challenging result comprising 21 inductive cases and 7 supporting lemmas completed without human intervention by both models independently. The tool is open-source and actively used in neurosymbolic AI research.

Keywords: Proof assistants, Rocq, Coq, LLMs, Formal verification, MCP, LSP, Neurosymbolic AI

1 Introduction

The integration of Large Language Models (LLMs) with formal verification tools represents a promising direction for automated theorem proving. While LLMs excel at pattern recognition and heuristic search over large code corpora, they lack the rigorous logical guarantees provided by proof assistants. Conversely, proof assistants like Rocq [1] ensure absolute correctness but require significant expert knowledge and manual effort.

rocq-robot addresses this challenge by implementing a *neurosymbolic architecture*: LLMs propose proof tactics based on learned patterns, while Rocq verifies each step’s correctness through its Language Server Protocol (LSP) interface [4]. This collaboration achieves both the creativity of neural systems and the soundness of symbolic reasoning.

Recent work has demonstrated LLM capabilities in proof synthesis [2, 3], but practical integration with proof assistants remains challenging. **rocq-robot** fills this gap by providing a production-ready tool with:

- **Structured MCP interface:** 20+ tools exposing proof state, tactic execution, and knowledge search
- **Project-aware management:** Multi-file workspace support with dependency tracking
- **Speculative execution:** Safe tactic exploration without file modifications
- **Error recovery:** Actionable feedback enabling LLM retry strategies

- **One-shot proof capability:** Demonstrated with both DeepSeek v4 and Claude Sonnet 4.5 independently completing type preservation proof (21 cases, 7 lemmas) in single autonomous executions
- **Production readiness:** Active use in research, comprehensive documentation, extensive test coverage

Availability: rocq-robot is open-source (MIT license), available at <https://github.com/scidonia/rocq-robot>, published on npm as `rocq-robot`, and actively maintained.

2 Tool Architecture

2.1 System Overview

rocq-robot implements a three-layer architecture connecting LLMs to Rocq through MCP:

1. **MCP Server Layer:** Exposes 20+ tools following the Model Context Protocol specification, enabling any MCP-compatible LLM client (Claude, OpenCode, custom agents) to interact with Rocq
2. **LSP Client Layer:** Communicates with rocq-lsp using Language Server Protocol for proof state queries, document synchronization, and diagnostic messages
3. **Workspace Manager:** Manages multi-file projects, detects project roots (`_CoqProject`, `_RocqProject`, `dune-project`), and tracks file dependencies

This architecture ensures separation of concerns: MCP handles LLM interaction, LSP manages proof state, and the workspace layer provides project context.

2.2 Core Tool Categories

rocq-robot organizes its 20+ tools into four categories aligned with theorem proving workflows:

Proof Interaction Tools

- `coq.focus`: Navigate to proof by name, retrieve goal state and bullet structure
- `coq.insert_tactic`: Insert tactics with automatic bullet prefix handling
- `coq.open_goals`: Query current proof obligations in human-readable format
- `coq.add_lemma`: Insert helper lemmas above target proofs
- `coq.reset_proof`: Restart failed proof attempts

Speculative Execution Tools Critical for LLM exploration without committing edits:

- `coq.try_tactic`: Execute tactics speculatively, return updated goals without file modification
- `coq.run_tactic`: Execute commands against specific proof states (Pétanque backend)
- `coq.get_state_at_pos`: Obtain opaque state identifiers for Pétanque queries

Knowledge Search Tools Enable LLMs to discover relevant lemmas and definitions:

- `coq_search`: Search environment by name or type pattern
- `coq_locate`: Find definition locations
- `coq_check_term`: Query term types
- `coq_about`: Retrieve symbol information
- `coq_require`: Speculatively import libraries

Workspace Management Tools Support multi-file development:

- `coq_check`: Force document verification
- `coq_apply_edit`: Apply text edits with LSP synchronization
- Automatic project root detection and dependency tracking

2.3 Implementation Details

`rocq-robot` is implemented in TypeScript (3,500+ LOC) using:

- `@modelcontextprotocol/sdk` for MCP server implementation
- `vscode-languageclient` for LSP communication with `rocq-lsp`
- `tsx` runtime for Node.js 20+ execution
- Comprehensive logging system for debugging and monitoring

The tool maintains internal state including:

- File version tracking for LSP synchronization
- Operation history for undo/redo functionality
- Proof cursor positions for focused interaction
- Project workspace metadata and dependencies

3 Usage Workflow

3.1 Installation and Setup

Installation requires three steps:

1. Install Node.js 20+ and Rocq 8.19+ (or Coq 8.19+)
2. Install `rocq-lsp`: `opam install rocq-lsp`
3. Install `rocq-robot`: `npm install -g rocq-robot`

Configure MCP client (e.g., Claude Desktop, OpenCode) to launch `rocq-robot`:

```
{
  "mcpServers": {
    "rocq-robot": {
      "command": "rocq-robot"
    }
  }
}
```

3.2 Autonomous Proof Development

LLMs interact with `rocq-robot` through natural language instructions translated to tool calls. A typical workflow:

1. **Focus on proof:** LLM calls `coq_focus("has_type_preservation")` to retrieve current goal state and proof context
2. **Explore tactics:** LLM uses `coq_try_tactic` to speculatively test tactics without file modifications
3. **Search knowledge:** LLM searches for relevant lemmas via `coq_search("substitution")` or `coq_locate`
4. **Insert tactics:** Once validated, LLM commits tactics via `coq_insert_tactic`
5. **Handle errors:** Failed tactics return diagnostic messages; LLM retries with corrections
6. **Add lemmas:** For complex goals, LLM inserts helper lemmas via `coq_add_lemma`

This workflow enables iterative refinement: the LLM proposes strategies, `rocq-robot` provides immediate verification, and the cycle continues until proof completion.

4 Case Study: Type Preservation Proof

We demonstrate `rocq-robot`'s effectiveness through an autonomous proof of type preservation for PCF with references—a challenging theorem requiring deep induction and auxiliary lemmas.

4.1 Problem Specification

The theorem states that well-typed expressions preserve types under evaluation:

$$\frac{\Gamma \vdash e : \tau \quad e \rightarrow e'}{\Gamma \vdash e' : \tau}$$

The proof requires:

- Handling 21 inductive cases (lambda, application, reference operations, etc.)
- Proving 7 supporting lemmas (substitution, weakening, store typing)
- Managing complex proof state with multiple nested goals

4.2 Autonomous Proof Process

We validated `rocq-robot`'s effectiveness by successfully completing the proof with *two different LLMs*: DeepSeek v4 and Claude Sonnet 4.5. Both models completed the proof autonomously in a *single execution* without human intervention, demonstrating the tool's robustness across different LLM architectures. The workflow for both models was identical:

1. **Main theorem structure:** LLM analyzed theorem statement and applied `induction` on the evaluation derivation
2. **Case analysis:** For each of 21 cases, LLM:
 - Used `coq_try_tactic` to explore case-specific tactics
 - Searched for applicable lemmas via `coq_search`

- Applied inversions, substitutions, and reconstructions
3. **Lemma discovery:** When blocked, LLM identified missing lemmas:
 - `substitution_preserves_typing`: Substitution maintains types
 - `weakening`: Adding hypotheses preserves typing
 - `store_weakening`: Extending stores preserves types
 4. **Helper lemma proofs:** LLM recursively proved each helper using same workflow
 5. **Error recovery:** Failed tactics triggered retries with diagnostic-informed corrections

Results: Both DeepSeek v4 and Claude Sonnet 4.5 independently completed the proof in single one-shot executions (approximately 2-3 hours of LLM processing time each), with zero human intervention. All 21 cases and 7 lemmas were formally verified by Rocq in both runs. This cross-model validation demonstrates that modern LLMs, regardless of their specific architecture, can autonomously complete complex formal verification tasks that traditionally required expert human guidance when equipped with appropriate tool interfaces.

4.3 Proof Statistics and Cross-Model Comparison

Statistics averaged across both successful runs:

- **Total proof size:** 450 lines of Rocq code
- **Tool calls:** ~850 total (~420 speculative, ~430 committed)
- **Search queries:** ~90 calls to `coq_search`, ~35 to `coq_locate`
- **Failed tactics:** ~65-70 (7-8% failure rate, all recovered via retry)
- **Auxiliary lemmas:** 7 lemmas totaling 180 lines
- **Processing time:** 2-3 hours per model

Both models discovered the same 7 auxiliary lemmas and handled the 21 cases in similar order, though with slight variations in tactic choices. This convergence suggests that `rocq-robot`'s structured interface effectively guides diverse LLMs toward correct proof strategies. The proof demonstrates `rocq-robot`'s capacity for robust, cross-model autonomous formal verification in realistic theorem proving scenarios.

5 Related Work

LLM-based theorem proving: Baldur [2] generates Coq proofs via retrieval-augmented LLMs but requires custom integration. LeanDojo [3] provides similar capabilities for Lean but uses a specialized architecture. `rocq-robot` uniquely offers a generic MCP interface compatible with any LLM client.

Proof assistant interfaces: `coq-lsp` [4] provides LSP support for Rocq, enabling editor integration. Earlier tools like CoqPIE [5] and Proof General [6] focused on IDE integration. `rocq-robot` builds atop `coq-lsp`, adding LLM-specific capabilities (speculative execution, structured state queries).

Interactive development environments: Recent IDEs integrate proof assistants with advanced features, but none target LLM interaction specifically. `rocq-robot` fills this gap by designing tools explicitly for autonomous AI agents.

6 Lessons Learned and Future Directions

6.1 Design Insights

Key lessons from `rocq-robot` development:

- **Speculative execution is critical:** LLMs require safe exploration without file commits; this drove our Pétanque integration
- **Error messages must be actionable:** Raw LSP diagnostics confuse LLMs; `rocq-robot` adds context and suggestions
- **Project awareness matters:** Real proofs span multiple files; workspace management is essential
- **Bullet handling is subtle:** Rocq’s proof structure requires automatic bullet prefix insertion for LLM usability

6.2 Future Work

Planned enhancements:

- **Proof analytics:** Collect metrics on tactic effectiveness, search patterns, and error recovery strategies
- **Multi-agent coordination:** Enable multiple LLM agents to collaborate on large proofs
- **Learning from proofs:** Extract successful proof patterns for fine-tuning LLMs
- **Interactive proof repair:** Detect invalid proofs after library updates and suggest fixes

7 Conclusion

`rocq-robot` demonstrates that neurosymbolic programming—combining LLM creativity with formal verification—is practical and effective for autonomous theorem proving. By providing a production-ready MCP interface to Rocq, the tool enables any LLM client to construct formally verified proofs through structured interaction, speculative execution, and knowledge search.

The successful autonomous proof of type preservation (21 cases, 7 lemmas) validates the architecture and shows that LLMs, when equipped with appropriate tools, can tackle realistic formal verification tasks. `rocq-robot` is open-source, actively maintained, and ready for use in both research and industrial applications.

Tool availability: <https://github.com/scidonia/rocq-robot>

References

- [1] The Coq Development Team. The Coq Proof Assistant. <https://coq.inria.fr/>, 2024.
- [2] First, E., Rabe, M.N., Ringer, T., Brun, Y.: Baldur: Whole-Proof Generation and Repair with Large Language Models. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) (2023)
- [3] Yang, K., Swope, A.M., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R., Anandkumar, A.: LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)

- [4] Gallego Arias, E.J., Fernández, R., Itzhaky, S., Charguéraud, A.: `coq-lsp`: A Modern Language Server for Coq. <https://github.com/ejgallego/coq-lsp>, 2022.
- [5] Roe, K.: CoqPIE: A Plugin for Integrated Development Environments. In: Conference on Intelligent Computer Mathematics (CICM) (2016)
- [6] Aspinall, D.: Proof General: A Generic Tool for Proof Development. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (2000)